

ENTERPRISE COMPUTING

Service-Oriented Architecture

LAB 3/W4

4TH YEAR - ENSIA

OBJECTIVES

- Build REST/SOAP Web Services using Spring Boot
- Configure an API Gateway (Spring Cloud Gateway)
- Implement orchestration using Spring Integration (ESB pattern)

VALIDATING “LAB 2” ASSIGNMENTS (A1)

Each student needs to demonstrate a working version of the minimal DMS service using Java spring boot on their local computer.

INDIVIDUAL EXERCISES

In this lab, a minimal DMS will be developed using the following web services:

Service	Port	Technology	Responsibility
D : Documents	8081	REST + JPA/H2	Add and list documents
M : Comments	8083	REST + JPA/H2	Add and list comments per document
C : Categories	8082	SOAP	Provide document categories
G : Gateway	8080	Spring Cloud Gateway	Hide D and M (Hide Port and addresses)
E : ESB	8084	Spring Integration	Orchestrate D + M

Task / Exercise C1 [55 Minutes] (Creating Web Services : REST Web Services) :

Part 1:

In the same way as :

https://www.dropbox.com/scl/fo/07plrjo7mw3r2ncbavnyr/Aff4zdoL2e4wFK3KW_eJYr0?rlkey=ldh7po856rhbgft8uq0peadw0&st=or3ikao7&dl=0

Create the Service D running on port 8081 where we can add a document by just providing a title only. Documents should be stored inside H2 store. Implement the following endpoints:

- [GET] documents/list
- [GET] documents/get/{id}
- [POST] documents/add

The code must be tested using POSTMAN.

Part 2:

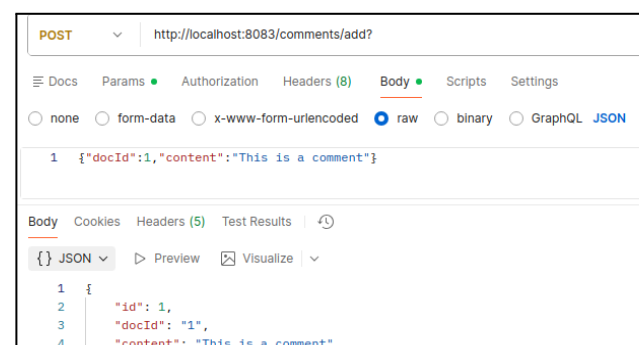
As a totally separate new project, Create a minimal REST web service M for comments running on port 8083. Create the following endpoints:

- [GET] comments/list/{docId}
- [POST] comments/add

Would merging Comments inside Documents improve or harm scalability? Cohesion ? coupling?

Task / Exercise C2 [15 Minutes] (Gateway) :

As a new project, Use the spring cloud gateway library to create a service that runs on port 8080, use the following application.yml file : (src/main/resources)



```

server:
  port: 8080
spring:
  application:
    name: gateway-service
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: documents
              uri: http://localhost:8081
              predicates:
                - Path=/api/documents/**
              filters:
                - RewritePath=/api/(?<segment>.*), /${segment}

            - id: comments
              uri: http://localhost:8083
              predicates:
                - Path=/api/comments/**
              filters:
                - RewritePath=/api/(?<segment>.*), /${segment}

```

- Discuss how to access the services via gateway.
- What problem does Gateway solve?
- Can the Gateway become a Single Point of Failure?
If yes, how would you mitigate that?

Task / Exercise C3 [15 Minutes] (ESB/Orchestration) :

Create a new project for the orchestration service (E) running on port 8084 that talks to services D and M to get data. The following dependencies should be used:

```

spring-boot-starter-integration
spring-boot-starter-web

```

You need to use IntegrationFlow with Message-driven orchestration architecture style

1. Implement the following endpoint :
 - [GET] /document/{id}
 It returns the document (from the service D) and comments from service M in the format :

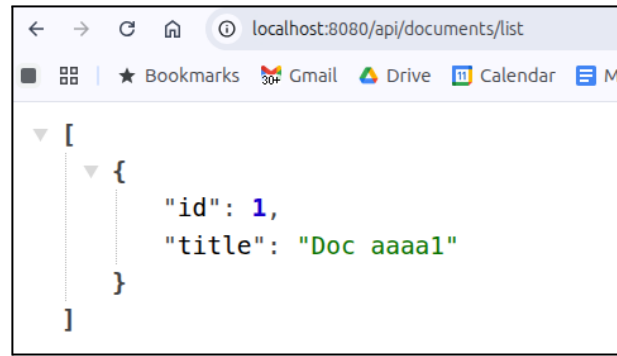

```
{ document: {...}, comments: [{...},...] }
```
2. What happens if the comments service goes down ? Prioritize the Availability as long as the document service is running.
3. Draw the architecture for this minimal version of the document management system

ASSIGNMENT

There is no assignment for next week, but students must finish all exercises (C1,C2,C3) in this worksheet on their own and make sure they present it next week running.

UPCOMING DELIVERABLES DEADLINES (Still Draft).

- [27 FEB] BACKEND INCREMENT 1 : INDIVIDUAL SUBMISSION
 - CAP + SOA + Gateway + ESB + Docker/K8 + S3 Storage
- [11 APR] FRONT-END PROTOTYPE SUBMISSION : GROUP PROJECT.
 - React App showing the UI components connected to Fake APIs.
- [25 APR] BACKEND INCREMENT 2 : INDIVIDUAL SUBMISSION
 - Distributed Data Storage + Async/Event's Arch + Auth/Security
- [16 MAY] INTEGRATION AND FINAL SUBMISSION : GROUP PROJECT.
 - E2E Testing
 - Use of DevOps
 - Integration of AI Services.
 - Demonstration of the :
 - Enterprise Attributes + Primary Functionalities.
 - Presentation Defending the Software Architecture and Design Decisions



OPTIONAL EXERCISES

Task / Exercise P1 :

- Change the code to have a universal unique Id for a document ?
- Hide completely d and m to be accessible only from the gateway
- Change the gateway to hide service E
- Why is orchestration centralized in ESB?

Task / Exercise P2 (SOAP/SOA) :

As another new separate project, we need to create a SOAP webservice to return the list of categories that runs on port 8082. The service endpoint is given below. **Test it using Postman.**

You can use this code:

```
@Endpoint
public class CategoryEndpoint {

    @PayloadRoot(namespace = "http://lab.com/categories",
        localPart = "GetCategoriesRequest")
    @ResponsePayload
    public GetCategoriesResponse getCategories() {

        GetCategoriesResponse response = new GetCategoriesResponse();
        response.getCategory().add("Finance");
        response.getCategory().add("HR");
        response.getCategory().add("Legal");

        return response;
    }
}
```

Task / Exercise P3 :

Explore the use of the following :

- spring-boot-starter-actuator
- filters for the Gateway..

Task / Exercise P4 :

If a document is deleted in D, what should happen to its comments in M? Design a consistency strategy.

Task / Exercise P5 :

Integrate external storage engine to use Supabase for storing documents whilst a noSQL database for storing the comments.

Task / Exercise P6 :

Why this implementation for the ESB/Orchestration is not recommended:

```
@RestController
public class AggregationController {
    private final RestTemplate restTemplate;
    public AggregationController(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }
    @GetMapping("/documents/{id}/full")
    public Map<String, Object> full(@PathVariable Long id) {

        Object doc =
            restTemplate.getForObject("http://localhost:8081/documents/" + id, Object.class);
        Object comments =
            restTemplate.getForObject("http://localhost:8083/comments/document/" + id, Object.class);

        return Map.of(
            "document", doc,
            "comments", comments
        );
    }
}
```